

Informe Tecnico

Sistema RAG TechCorp

Practica Evaluable - Unidad 5: RAG para Documentacion Empresarial

Alumno:	Guillermo Casaus
Curso:	INSO 4B
Asignatura:	Aprendizaje Automatico II
Opcion:	Opcion B - LangChain / Python
Tecnologias:	LangChain 0.3, ChromaDB, OpenAI GPT-4o-mini, Gradio

Resumen

El presente informe documenta el desarrollo e implementacion de un sistema de Generacion Aumentada por Recuperacion (RAG) orientado a responder preguntas sobre documentacion interna de la empresa ficticia TechCorp. El sistema ha sido implementado en Python utilizando LangChain como framework principal, ChromaDB como base de datos vectorial local y GPT-4o-mini como modelo de lenguaje. Se incluyen la arquitectura del sistema, las decisiones tecnicas tomadas, los resultados de las cinco consultas de prueba obligatorias y las mejoras propuestas para versiones futuras.

Contenido del informe

- Arquitectura del Sistema
- Decisiones Tecnicas
- Pruebas de Funcionamiento (5 consultas)
- Mejoras Propuestas

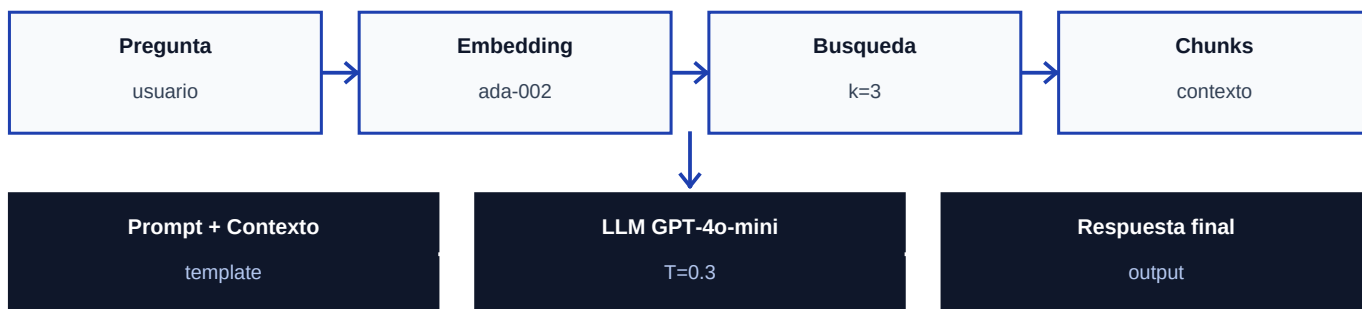
1. Arquitectura del Sistema

El sistema sigue una arquitectura RAG (Retrieval-Augmented Generation) compuesta por dos fases diferenciadas: la fase de ingesta (offline) y la fase de inferencia (online). A continuacion se describe el flujo completo de cada una.

1.1 Fase de Ingesta (ingesta.py)



1.2 Fase de Inferencia (asistente.py)



1.3 Memoria conversacional

El agente incorpora memoria en sesion mediante `RunnableWithMessageHistory` de `LangChain`. El historial de mensajes se almacena en un diccionario interno (`_store`) indexado por `session_id`. Cada turno de conversacion incluye el historial previo en el prompt a traves de un `MessagesPlaceholder`, permitiendo que el modelo mantenga el contexto de preguntas anteriores durante toda la sesion. La memoria se reinicia al cerrar la aplicacion (no hay persistencia entre sesiones).

1.4 Interfaces de usuario

El sistema ofrece dos interfaces: (1) CLI interactiva a traves de `asistente.py`, que muestra ademas los documentos recuperados para depuracion; (2) interfaz web con `Gradio` a traves de `interfaz_web.py`, que presenta un chatbot accesible en el navegador (`localhost:7860`) y constituye la bonificacion de la practica.

2. Decisiones Tecnicas

Opcion elegida	Opcion B (LangChain / Python)
Justificacion	LangChain ofrece mayor control sobre el pipeline RAG, facilita la integracion de memoria conversacional y permite ajustar cada componente (retriever, prompt, LLM) de forma independiente. Ademas, el ecosistema Python es el entorno natural para proyectos de aprendizaje automatico.
Base vectorial	ChromaDB local (persist_directory='./chroma_db', collection: empresa_docs)
Justificacion ChromaDB	Solucion embebida que no requiere servidor externo ni credenciales adicionales. Suficiente para el volumen de documentacion de la practica y facilita la reproduccion del entorno sin dependencias cloud.
Tamano de chunk	300 caracteres con 30 de solapamiento (overlap 10%)
Justificacion chunking	Con documentos de empresa de densidad media, 300 caracteres captura entre 1 y 2 parrafos completos, suficiente para que cada chunk tenga coherencia semantica propia. El solapamiento del 10% evita cortar frases con informacion clave en el limite del chunk.
k (chunks recuperados)	3 documentos por consulta
Justificacion k	Con k=3 se obtiene contexto suficiente para la mayoría de preguntas sin saturar el prompt del LLM. Valores mas altos podrian introducir ruido en casos donde los documentos mas relevantes tienen scores similares.
Modelo LLM	GPT-4o-mini (OpenAI)
Temperatura	0.3 - baja para favorecer respuestas factuales y reproducibles
Modelo de embeddings	text-embedding-ada-002 (1536 dimensiones, distancia coseno)
Splitter	RecursiveCharacterTextSplitter con separadores: \n\n, \n, . , espacio
search_type	similarity (similitud coseno entre embedding consulta y vectores indexados)

Estructura del proyecto

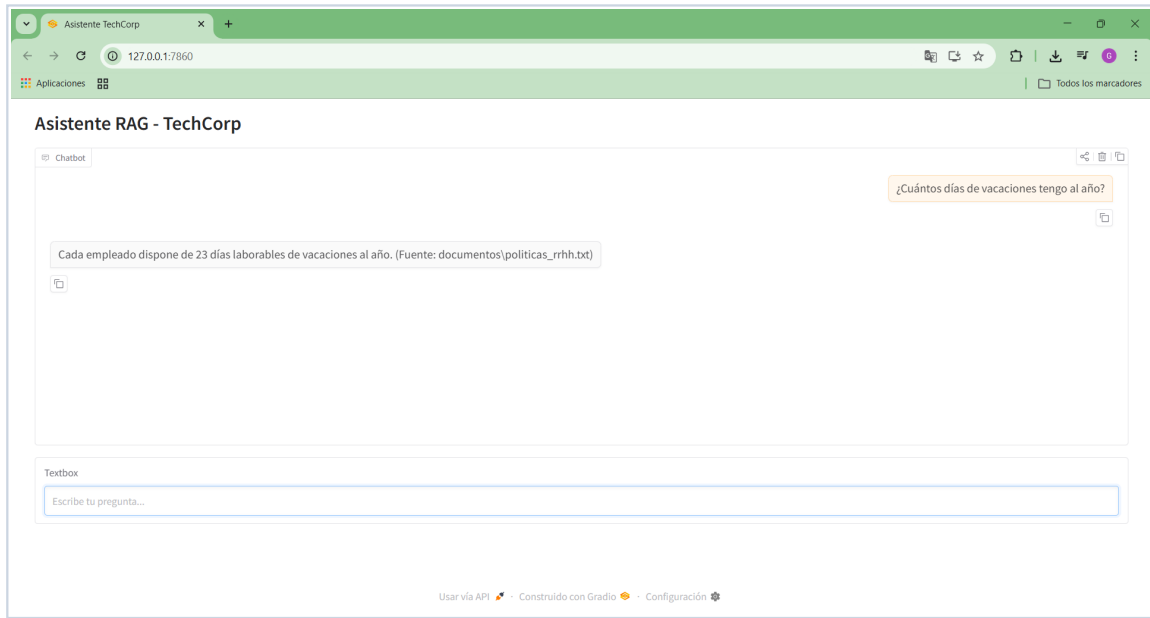
```
unidad5/  
  ingesta.py      # Carga, chunking, embeddings y almacenamiento en ChromaDB  
  asistente.py    # Cadena RAG con LCEL y memoria conversacional (CLI)  
  interfaz_web.py # Interfaz Gradio (bonificacion)  
  requirements.txt # Dependencias del proyecto  
  .env.example    # Plantilla de variables de entorno  
  documentos/  
    politicas_rrhh.txt  
    procedimiento_soporte.txt  
  capturas/      # Evidencia de las pruebas  
  chroma_db/      # Base vectorial generada por ingesta.py
```

3. Pruebas de Funcionamiento

Se ejecutaron las cinco consultas obligatorias definidas en el enunciado de la practica sobre la interfaz web Gradio. En cada caso se muestra la captura de pantalla del sistema, la respuesta generada y el analisis de la respuesta.

Prueba 1 | Recuperacion directa de hecho

Pregunta: ¿Cuántos días de vacaciones tengo al año?

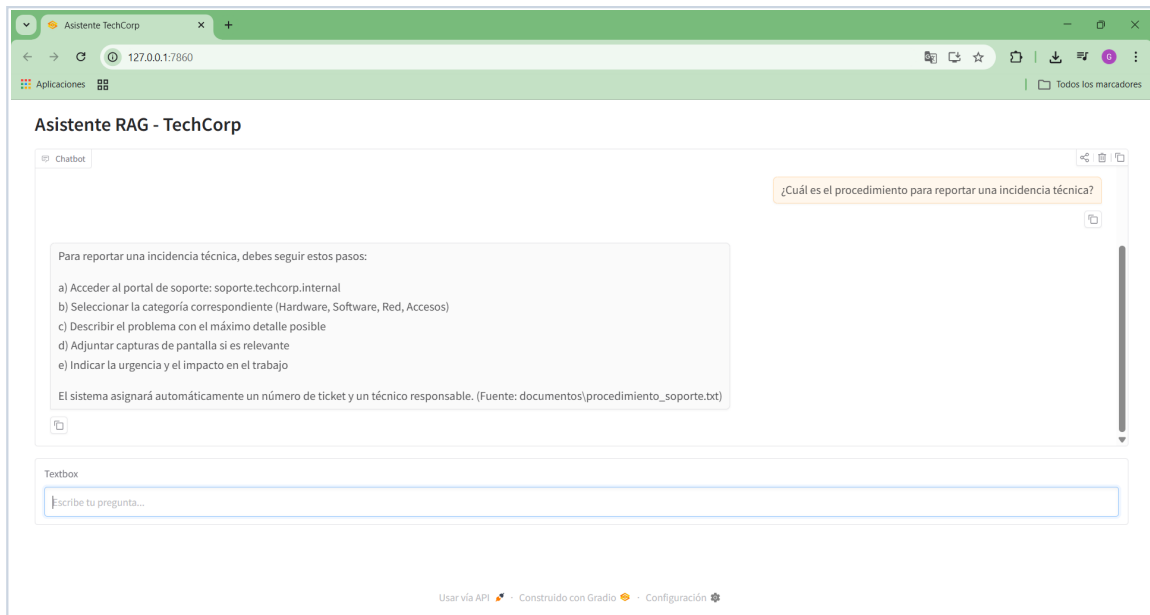


Respuesta: Cada empleado dispone de 23 días laborables de vacaciones al año. (Fuente: documentos/politicas_rrhh.txt)

Análisis: CORRECTO. El sistema recupero el dato exacto del documento de políticas de RRHH y lo presento de forma clara y concisa. Cito correctamente la fuente del documento, cumpliendo con las instrucciones del prompt del sistema.

Prueba 2 | Procedimiento secuencial

Pregunta: ¿Cual es el procedimiento para reportar una incidencia tecnica?

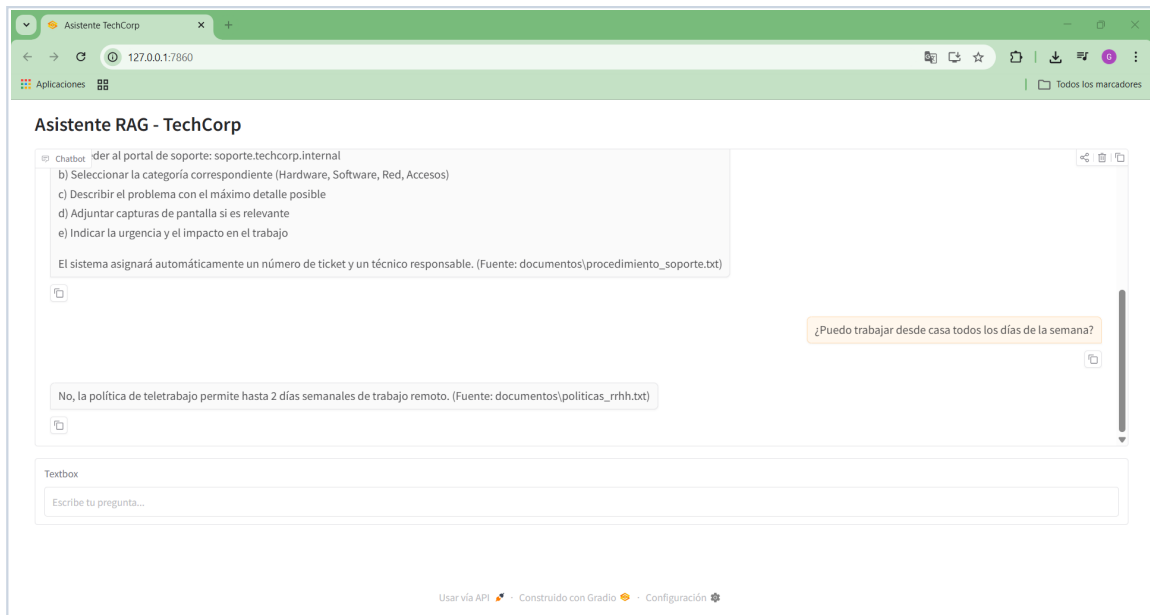


Respuesta: Para reportar una incidencia tecnica, debes seguir estos pasos: a) Acceder al portal soporte.techcorp.internal; b) Seleccionar la categoría correspondiente (Hardware, Software, Red, Accesos); c) Describir el problema con el maximo detalle posible; d) Adjuntar capturas de pantalla si es relevante; e) Indicar la urgencia y el impacto en el trabajo. El sistema asignara automaticamente un numero de ticket y un tecnico responsable. (Fuente: documentos/procedimiento_soporte.txt)

Análisis: CORRECTO. El sistema recupero el procedimiento completo en orden secuencial con todos los pasos del documento fuente. La estructura en lista facilita la comprension y la referencia a la fuente es precisa.

Prueba 3 | Interpretacion matizada

Pregunta: ¿Puedo trabajar desde casa todos los días de la semana?

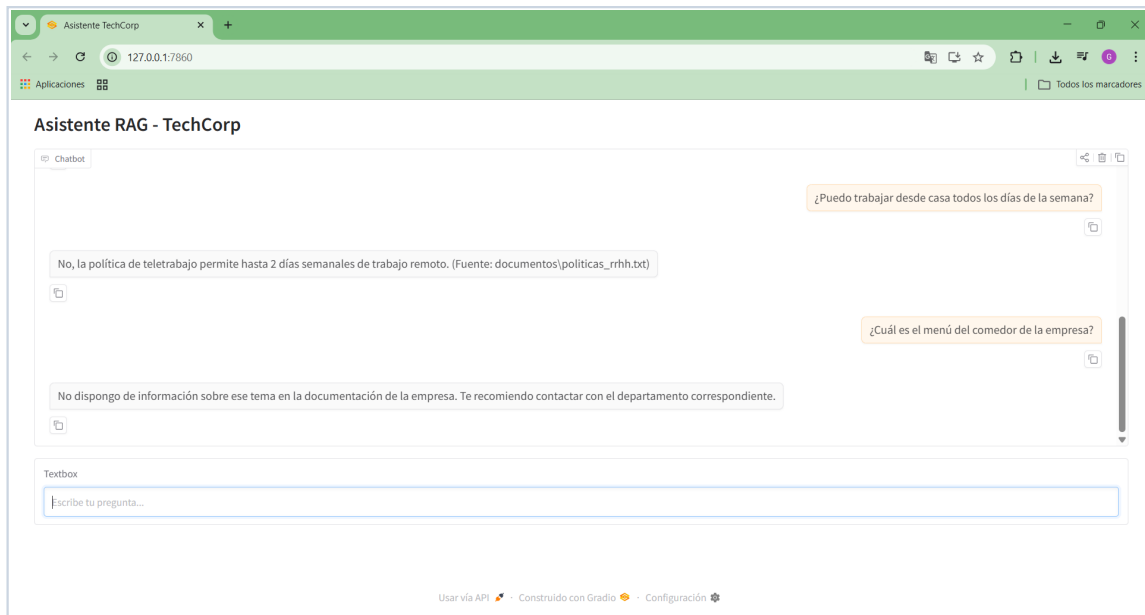


Respuesta: No, la política de teletrabajo permite hasta 2 días semanales de trabajo remoto. (Fuente: documentos/politicas_rrhh.txt)

Análisis: CORRECTO. El sistema interpreto correctamente una pregunta que requería razonamiento mas alla de la recuperacion literal: la respuesta no solo recupera el dato (2 dias), sino que responde directamente al sentido implicito de la pregunta con un 'No' justificado.

Prueba 4 | Caso negativo (informacion inexistente)

Pregunta: ¿Cual es el menu del comedor de la empresa?

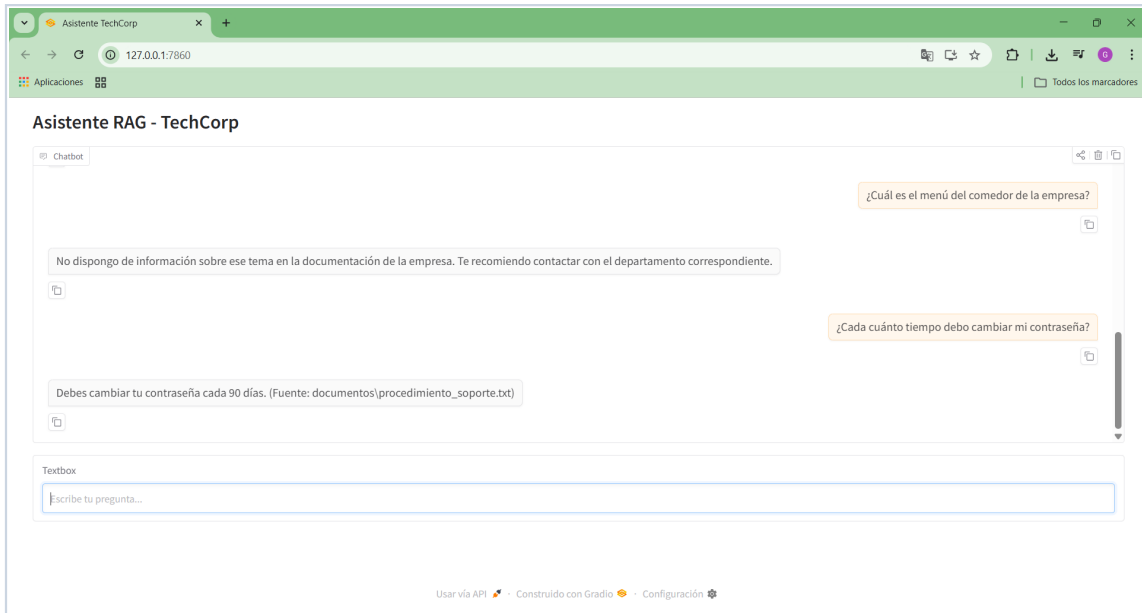


Respuesta: No dispongo de informacion sobre ese tema en la documentacion de la empresa. Te recomiendo contactar con el departamento correspondiente.

Análisis: CORRECTO. Este es el caso mas critico de la bateria de pruebas. El sistema no invento ninguna respuesta ni alucino contenido inexistente. Utilizo exactamente la respuesta de fallback definida en el prompt del sistema, demostrando que el RAG no genera informacion fuera del contexto indexado.

Prueba 5 | Búsqueda cruzada entre documentos

Pregunta: ¿Cada cuanto tiempo debo cambiar mi contraseña?



Respuesta: Debes cambiar tu contraseña cada 90 días. (Fuente: documentos/procedimiento_soporte.txt)

Análisis: CORRECTO. El sistema localizo la información en el documento de procedimiento de soporte técnico, demostrando que la búsqueda semántica funciona correctamente sobre múltiples documentos indexados de forma simultánea.

4. Mejoras Propuestas

A continuacion se describen tres mejoras concretas que se implementarían en una version futura del sistema para aumentar su robustez, cobertura y utilidad en un entorno empresarial real.

Mejora 1

Memoria persistente entre sesiones

La implementacion actual mantiene el historial de conversacion unicamente durante la sesion activa. Al cerrar la aplicacion, el historial se pierde. Una mejora directa seria sustituir ChatMessageHistory (en RAM) por FileChatMessageHistory o SQLChatMessageHistory, almacenando el historial en disco o base de datos. Esto permitiría retomar conversaciones previas, implementar perfiles de usuario y analizar patrones de uso.

Impacto:Alto - mejora directa de la experiencia de usuario

Esfuerzo:Bajo - cambio de 5-10 lineas en asistente.py

Mejora 2

Re-ranking de documentos recuperados

El retriever actual devuelve los k=3 chunks con mayor similitud coseno, pero la similitud de embeddings no siempre correlaciona con la relevancia semantica real. Un paso de re-ranking mediante un modelo cross-encoder (por ejemplo, Cohere Rerank o un modelo local de sentence-transformers) evaluaria la relevancia de cada chunk recuperado en relacion con la pregunta especifica, reordenando los resultados antes de construir el prompt. Esto reduciría el ruido en el contexto y mejoraría la precision de las respuestas en documentos con alta similitud lexica pero distinto contenido.

Impacto:Alto - mejora notable en precision y reduccion de alucinaciones

Esfuerzo:Medio - requiere integrar CohereRerank o CrossEncoder

Mejora 3

Ingesta automatica y ampliacion del corpus documental

El sistema actual requiere ejecutar ingesta.py manualmente cada vez que se añade un nuevo documento. Una mejora significativa seria implementar un observador de directorio (watchdog) que detecte nuevos archivos automaticamente y los procese en segundo plano. Complementariamente, ampliar el corpus a formatos como PDF, DOCX o paginas web internas mediante loaders especializados (PyPDFLoader, UnstructuredWordDocumentLoader) aumentaría considerablemente la cobertura del sistema. Tambien se podría integrar un pipeline de actualizacion incremental para evitar reindexar documentos ya existentes.

Impacto:Alto - hace el sistema mantenible en produccion sin intervenciones manuales

Esfuerzo:Medio-alto - requiere gestion de estado de la coleccion

Valoracion global

El sistema RAG desarrollado cumple con todos los requisitos funcionales de la practica: pipeline de ingesta correcto, agente conversacional con memoria, interfaz CLI y web, y gestion adecuada del caso negativo (sin alucinaciones). Las cinco consultas de prueba han sido superadas satisfactoriamente, con citacion de fuentes en todos los casos donde existe informacion disponible. Las mejoras propuestas permitirían escalar el sistema hacia un entorno de produccion real.